

FinSuite

An AI-Assisted Personal Finance and Live Stock Market Analysis Platform

Internship Project Report

submitted as part of the internship on

AI in Finance

Submitted by

Purva Gajanan Kakade

Under the guidance of

Prof. Ashish Singh

Shailesh J. Mehta School of Management (SJMSOM)

Indian Institute of Technology Bombay

October 2026

Declaration

I hereby declare that the project report entitled “FinSuite: An AI-Assisted Personal Finance and Live Stock Market Analysis Platform” is a record of the work carried out by me during my internship under the guidance of Prof. Ashish Singh, Shailesh J. Mehta School of Management, IIT Bombay. The work presented here is original to the best of my knowledge, and all external sources, libraries and data providers used have been duly acknowledged.

Purva Gajanan Kakade

Date: _____ Place: _____

Signature: _____

Acknowledgement

I would like to express my sincere gratitude to Prof. Ashish Singh, Shailesh J. Mehta School of Management, IIT Bombay, for his guidance, encouragement and valuable feedback throughout this internship. His insights on the application of artificial intelligence and data analysis in finance shaped the direction of this project.

I am also thankful to the maintainers of the open-source tools on which this work is built — FastAPI, pandas, NumPy, scikit-learn, yfinance and Chart.js — and to my family and friends for their constant support.

Purva Gajanan Kakade

Abstract

Most individuals make important financial decisions — whether to buy a stock, whether a loan is affordable, how much to save every month for a goal — without access to the analytical tools that banks and analysts use. This project presents **FinSuite**, a full-stack web application that brings those tools together in a single, free and easy-to-use console. FinSuite combines sixteen modules: financial-ratio calculators, a personal finance report, a rule-based finance assistant, loan and credit-risk screening, investment and company-performance scoring, a multi-company portfolio screener, goal and retirement planning, a finance glossary, a live stock-market workspace, growth and target planning, Indian capital-gains tax estimation, and a live watchlist with price alerts.

The system is implemented as a Python **FastAPI** backend exposing 29 documented REST endpoints and a lightweight HTML/JavaScript frontend. A key contribution is a key-free live market-data layer: any listed company, ETF or index worldwide can be searched by name or ticker, and the platform returns live quotes, daily OHLCV history and company-specific news, using Yahoo Finance (via the `yfinance` library) and Google News RSS. The layer handles symbol resolution (company names, bare NSE/BSE tickers), short-lived caching to avoid rate limiting, market holidays, and relevance filtering of news headlines.

The project also includes a machine-learning pipeline for next-day closing-price prediction based on engineered technical features (moving averages, daily return, volatility and RSI). Evaluated on two years of real data for Reliance Industries, TCS and Infosys, Linear Regression achieved a mean absolute error of ₹12.03, ₹34.27 and ₹17.58 respectively — no better than a naive “tomorrow equals today” baseline — while Random Forest performed worse, and neither model predicted the direction of the next move better than chance. These results are consistent with the weak-form efficient-market hypothesis and highlight the importance of honest baselines when applying AI to financial prediction.

Keywords: financial analytics, FastAPI, live market data, `yfinance`, credit risk, SIP planning, capital gains tax, machine learning, stock price prediction, efficient market hypothesis.

Table of Contents

Declaration.....	2
Acknowledgement	3
Abstract.....	4
List of Abbreviations	7
Chapter 1: Introduction.....	8
1.1 Background.....	8
1.2 Problem Statement.....	8
1.3 Objectives	8
1.4 Scope	9
1.5 Organisation of the Report	9
Chapter 2: Background and Literature Review	10
2.1 Financial Ratio Analysis.....	10
2.2 Credit Scores and Repayment Capacity	10
2.3 Time Value of Money and SIP Planning.....	10
2.4 Capital Gains Taxation on Listed Equity in India	11
2.5 Technical Indicators	11
2.6 Machine Learning for Price Prediction	11
2.7 Sources of Market Data	12
Chapter 3: System Requirements and Design	13
3.1 Functional Requirements.....	13
3.2 Non-Functional Requirements.....	13
3.3 System Architecture	13
3.4 Technology Stack	14
3.5 Project Structure	14
3.6 API Design	15
3.7 Request Flow for Live Market Data	16

Chapter 4: Implementation	17
4.1 Backend Foundation	17
4.2 Financial Calculator Modules.....	17
4.3 Live Market-Data Layer	21
4.4 Frontend.....	23
Chapter 5: Machine-Learning Experiment: Next-Day Price Prediction	26
5.1 Pipeline	26
5.2 Experimental Setup	26
5.3 Results	27
5.4 Discussion.....	27
5.5 Implications	28
Chapter 6: Testing and Validation.....	29
6.1 Approach	29
6.2 Representative Test Cases	29
6.3 Defects Found and Resolved	30
Chapter 7: Limitations and Future Work.....	31
7.1 Limitations.....	31
7.2 Future Work.....	31
Chapter 8: Conclusion	32
References	33
Appendix A: API Reference.....	34
Appendix B: Installation and Running	34
Appendix C: Application Home Screen	35

List of Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
CAGR	Compound Annual Growth Rate
EMI	Equated Monthly Instalment
ETF	Exchange-Traded Fund
LTCG / STCG	Long-Term / Short-Term Capital Gains
MA7 / MA30	7-day / 30-day Moving Average
MAE / RMSE	Mean Absolute Error / Root Mean Squared Error
ML	Machine Learning
NSE / BSE	National Stock Exchange / BSE (Bombay Stock Exchange)
OHLCV	Open, High, Low, Close, Volume
REST	Representational State Transfer
ROA / ROE	Return on Assets / Return on Equity
RSI	Relative Strength Index
RSS	Really Simple Syndication (news feed format)
SIP	Systematic Investment Plan
TTL	Time To Live (cache expiry)

Chapter 1: Introduction

1.1 Background

Financial literacy and access to analysis tools remain uneven. Professional investors and lenders rely on financial ratios, credit-risk bands, discounted-cash-flow style planning and live market data to make decisions, while students, first-time investors and small savers often rely on tips, rules of thumb or fragmented websites. Many free tools cover only one task (an EMI calculator, a SIP calculator, a stock quote page) and rarely explain **why** a result came out the way it did.

At the same time, the cost of building such tools has fallen sharply. Open-source web frameworks, scientific Python libraries and freely accessible market data make it possible for a single developer to build an integrated analytics platform. The “AI in Finance” internship provided the context to explore this: how far can a carefully engineered, transparent, free application go in giving ordinary users professional-grade financial analysis?

1.2 Problem Statement

To design and build an integrated, free, web-based finance platform that (a) performs the core analyses an individual investor or borrower needs, each with a clear explanation of the underlying formula; (b) provides reliable live market data and relevant news for **any** listed company without requiring paid API keys; and (c) critically evaluates whether machine-learning models can meaningfully predict short-term stock prices.

1.3 Objectives

- Implement validated calculators for financial ratios, personal finance health, credit and loan risk, investment scoring, goal/retirement planning, emergency funds, growth/target planning and Indian capital-gains tax.
- Expose every calculation through a documented REST API so the logic is reusable and testable independently of the user interface.
- Build a live market workspace with company search across exchanges, live quotes, daily history with charts, and company-specific news — with no API key or signup.
- Provide a live watchlist with automatic refresh and price alerts.
- Build and evaluate an ML pipeline (feature engineering, Linear Regression, Random Forest) for next-day price prediction against an honest naive baseline.

- Test the complete system end-to-end in a real browser and document defects found and fixed.

1.4 Scope

The platform targets individual users in India (Indian Rupee formatting, NSE/BSE-first search, Indian tax rules) while supporting global markets (US, Korea, Japan, Europe) for quotes and news. It is an educational and decision-support tool; it does not execute trades, store personal data, or constitute investment advice.

1.5 Organisation of the Report

Chapter 2 reviews the financial and technical concepts used. Chapter 3 presents requirements and system design. Chapter 4 describes the implementation of each module, with emphasis on the live market-data layer. Chapter 5 reports the machine-learning experiment. Chapter 6 covers testing and the defects resolved. Chapter 7 discusses limitations and future work, and Chapter 8 concludes. Appendices list the API, setup instructions and project files.

Chapter 2: Background and Literature Review

2.1 Financial Ratio Analysis

Ratio analysis condenses financial statements into comparable indicators of profitability, efficiency and leverage (Brealey, Myers & Allen, 2020). FinSuite uses the following standard definitions:

$$ROE = Net\ Income \div Shareholders'\ Equity \times 100$$

$$ROA = Net\ Income \div Total\ Assets \times 100$$

$$Debt\ Ratio = Total\ Liabilities \div Total\ Assets$$

$$Profit\ Margin = Net\ Income \div Revenue \times 100$$

$$Revenue\ Growth = (Revenue_t - Revenue_{t-1}) \div Revenue_{t-1} \times 100$$

Commonly used thresholds — ROE above 15–20%, ROA above 10%, profit margin above 15% — are used as scoring cut-offs in the investment and portfolio modules. These are heuristics; appropriate levels differ by industry.

2.2 Credit Scores and Repayment Capacity

Indian credit bureaus such as TransUnion CIBIL report scores in the range 300–900, with scores of 750 and above generally regarded as favourable by lenders. Lenders also assess repayment capacity through the ratio of EMI obligations to income (often called FOIR); a burden above roughly 40–50% of income is widely treated as a risk signal. FinSuite combines both: the credit score sets a base risk band and an EMI/income ratio above 40% raises the risk by one level.

2.3 Time Value of Money and SIP Planning

Goal planning requires inflating today's target to its future cost and solving for the monthly investment (an annuity-due, since SIP instalments are invested at the start of each month):

$$FV_goal = Target \times (1 + \pi)^n$$

$$SIP = (FV_goal - Savings \times (1 + r)^n) \div \{ [((1 + i)^m - 1) \div i] \times (1 + i) \}$$

$$where\ i = r \div 12\ and\ m = 12n$$

where π is annual inflation, r the expected annual return and n the number of years. The emergency-fund module follows the common guideline of holding six months of expenses in liquid savings.

2.4 Capital Gains Taxation on Listed Equity in India

Following the Union Budget of July 2024, gains on listed equity shares held for twelve months or less are taxed as short-term capital gains at 20% (Section 111A of the Income-tax Act), and gains on holdings beyond twelve months as long-term capital gains at 12.5% on the amount exceeding ₹1.25 lakh per financial year (Section 112A). A 4% health and education cess applies on the tax. FinSuite’s tax estimator implements these rules in simplified form (surcharge, grandfathering and set-off of losses are excluded).

2.5 Technical Indicators

Technical analysis summarises price behaviour into indicators. The project uses simple moving averages over 7 and 30 trading days, the daily return $(P_t - P_{t-1})/P_{t-1}$, 21-day rolling volatility (standard deviation of daily returns), and Wilder’s Relative Strength Index (Wilder, 1978):

$$RSI = 100 - 100 \div (1 + RS), \quad RS = \text{average gain} \div \text{average loss over 14 days}$$

RSI above 70 is conventionally read as “overbought” and below 30 as “oversold”.

2.6 Machine Learning for Price Prediction

A large body of work applies regression, tree ensembles and neural networks to stock-price forecasting. Random Forests (Breiman, 2001) average many decorrelated decision trees and are robust to noisy features, while linear regression offers interpretability. However, the efficient market hypothesis (Fama, 1970) holds that, in its weak form, past prices are already reflected in the current price, so models built only on price history should not systematically beat a random-walk forecast. A frequent pitfall in student and practitioner studies is reporting low error values without comparing them to the trivial “tomorrow equals today” forecast; this project explicitly includes that baseline (Chapter 5).

2.7 Sources of Market Data

The original version of the application used Alpha Vantage, whose free tier requires an API key and allows only 25 requests per day — insufficient for a watchlist that refreshes every 30 seconds. Table 2.1 compares the options.

Table 2.1: Comparison of market-data and news sources

Source	Key needed	Coverage	Limits / caveats
Alpha Vantage (free)	Yes	Global equities	25 requests/day; limited Indian coverage
Yahoo Finance via yfinance	No	Global equities, ETFs, indices, search	Unofficial; throttles bursts — mitigated by caching
Google News RSS	No	Worldwide news, 7-day window filter	Full-text matches need relevance filtering

Chapter 3: System Requirements and Design

3.1 Functional Requirements

- FR1 — Compute financial ratios, health scores and personal-finance reports with explanatory tips.
- FR2 — Screen loan applicants and customers by credit score and EMI burden.
- FR3 — Score single companies and portfolios of companies as BUY / HOLD / AVOID (SELL).
- FR4 — Plan goals (SIP required, year-wise projection) and assess emergency-fund adequacy.
- FR5 — Search any listed company by name or ticker and show its live quote, daily OHLCV history, chart and news.
- FR6 — Maintain a watchlist with automatic refresh and price alerts.
- FR7 — Estimate capital-gains tax, compare SIP vs lump-sum, analyse 52-week range and target/stop-loss plans.
- FR8 — Provide a glossary/assistant explaining finance terms.

3.2 Non-Functional Requirements

- **No paid keys or signup** — every data source must be free and keyless.
- **Responsiveness** — cached responses in under a second; live fetches typically 1–3 seconds.
- **Robustness** — clear error messages for unknown symbols, upstream outages and invalid input; graceful handling of market holidays.
- **Transparency** — every tool shows or documents its formula; the API is self-documented (OpenAPI/Swagger).
- **Portability** — runs on any machine with Python 3.11+; no database or build step.

3.3 System Architecture

FinSuite follows a classic client–server architecture (Figure 3.1). The browser loads a single HTML page whose JavaScript calls the FastAPI server over JSON/HTTP. The server holds all business logic: pure calculator functions, the market-data layer (which in turn calls Yahoo Finance and Google News over HTTPS) and a glossary. The ML pipeline lives in a separate service package and is currently used for offline experiments.

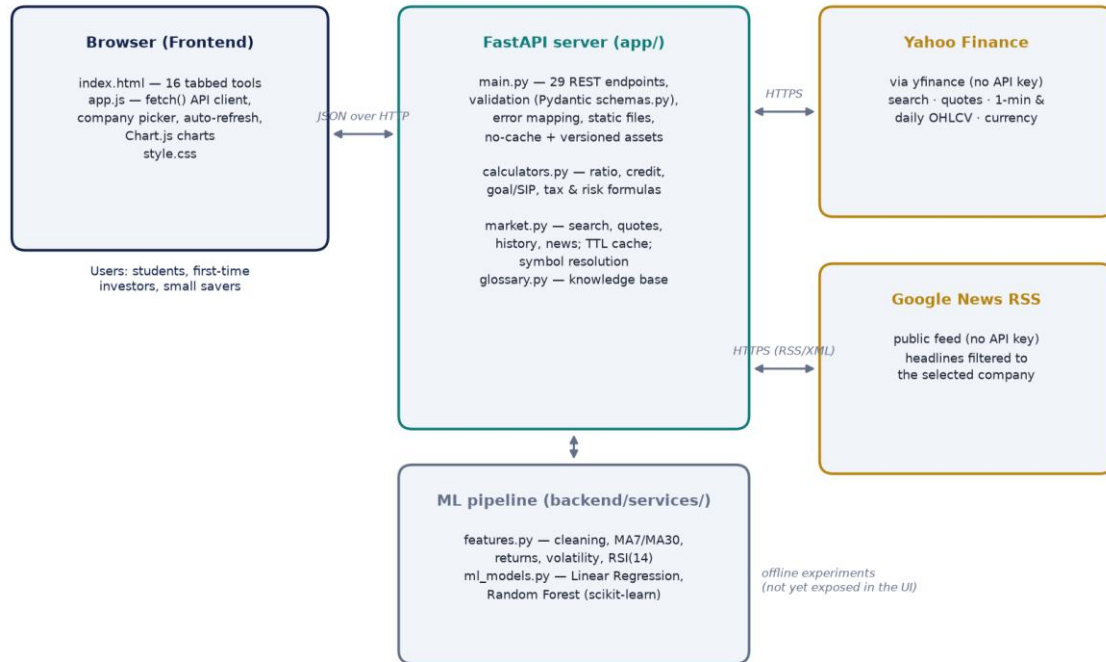


Figure 3.1: High-level architecture of FinSuite

3.4 Technology Stack

Table 3.1: Technology stack

Layer	Technology	Purpose
Backend framework	FastAPI 0.14x on Uvicorn (Python 3.13)	REST API, validation, OpenAPI docs, static files
Validation	Pydantic 2	Typed request schemas; automatic 422 errors
Data processing	pandas 3, NumPy 2	Time-series handling, feature engineering
Machine learning	scikit-learn 1.9	Linear Regression, Random Forest, metrics
Market data	yfinance 1.7, requests	Search, quotes, OHLCV history; Google News RSS
Frontend	HTML5, CSS3, vanilla JavaScript	16-tab single-page console
Charts	Chart.js 4.4	Price, projection and score charts

3.5 Project Structure

Table 3.2: Main source files

Path	Responsibility
app/main.py	FastAPI application: 29 routes, error mapping, caching headers, serves the frontend
app/calculators.py	Pure, side-effect-free financial formulas (modules 01–09, 12, 13)
app/market.py	Live data layer: search, symbol resolution, quotes, history, news, TTL cache
app/schemas.py	Pydantic request models for every POST endpoint

app/glossary.py	Knowledge-centre question/answer pairs
backend/services/features.py	Data cleaning and technical feature engineering
backend/services/ml_models.py	Linear Regression and Random Forest training/evaluation
backend/services/simulate.py	Synthetic OHLCV generator with injected errors (for practice/cleaning)
frontend/index.html, app.js, style.css	User interface

3.6 API Design

Calculators use POST with a JSON body validated by a Pydantic schema; data lookups use GET with query parameters. Validation failures return HTTP 422, an unknown symbol returns 404 with guidance, and an upstream data-provider failure returns 502. Endpoints are grouped by tags that match the sixteen UI sections, so the interactive documentation at /docs mirrors the application (Figure 3.2). The full list is in Appendix A.

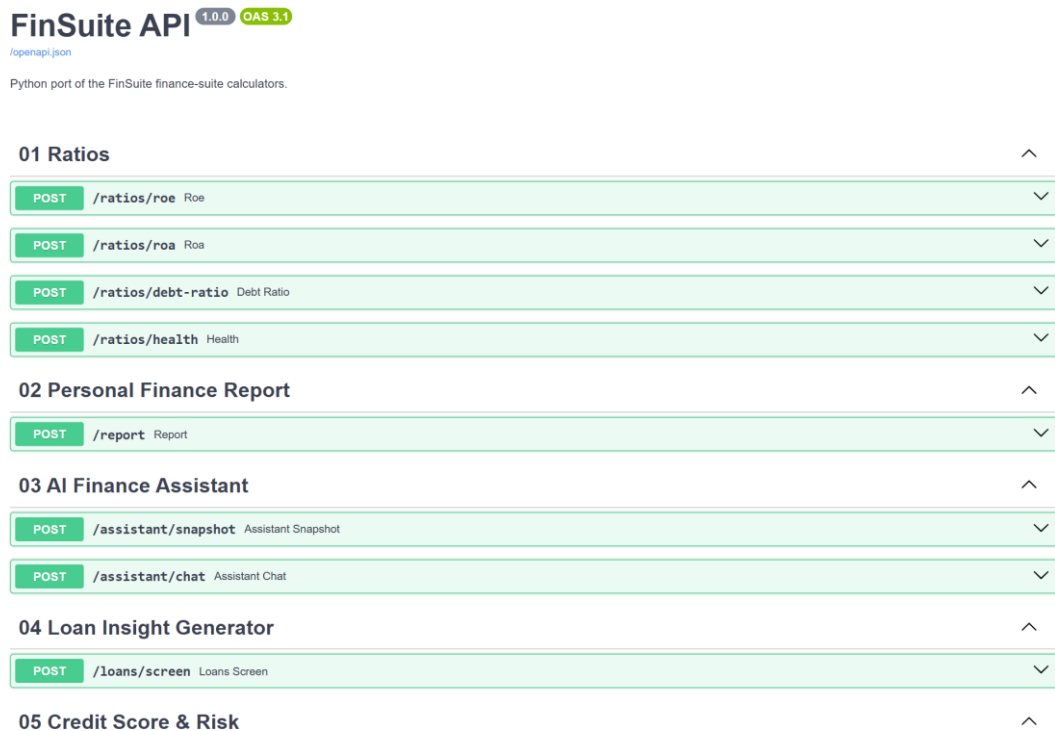
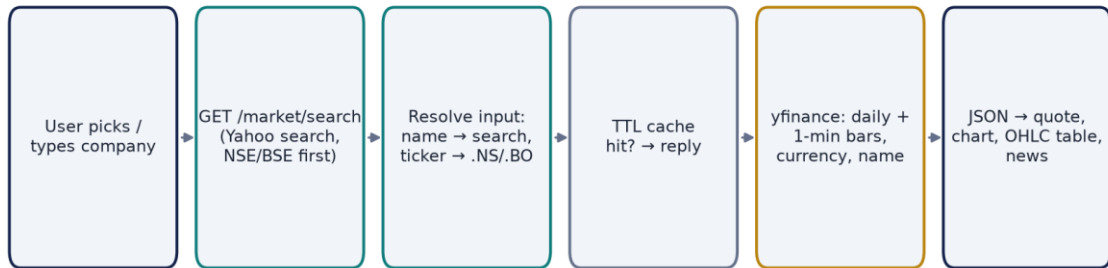


Figure 3.2: Auto-generated interactive API documentation (Swagger UI at /docs)

3.7 Request Flow for Live Market Data



Stale replies are discarded in the browser, so switching companies quickly never shows old data.

Figure 3.3: Flow of a company lookup from the picker to the rendered quote, chart, table and news

Chapter 4: Implementation

4.1 Backend Foundation

The FastAPI application registers CORS middleware, mounts the frontend as static files and wraps calculator calls in a helper that converts `ValueError` (e.g. division by zero equity) into HTTP 422 responses with a readable message. A small middleware sets `Cache-Control: no-cache` on the page and assets, and the root route stamps `app.js` and `style.css` URLs with their file-modification time (`?v=...`), so a browser can never combine a new page with an outdated cached script. The server can be started with `uvicorn app.main:app` or simply by running `python app/main.py`.

4.2 Financial Calculator Modules

All calculators are pure functions in `app/calculators.py`, which makes them deterministic and easy to test. Table 4.1 summarises the decision rules; the subsections explain the more involved ones.

Table 4.1: Calculator modules and decision rules

Module	Inputs	Logic / output
01 Ratios	Net income, equity, assets, liabilities	ROE, ROA, debt ratio; health: ROE > 20 Excellent, ≥ 10 Good, else Weak
02 Finance report	Income, expenses, savings, debt, equity	Net profit, ROE, ROA, current ratio, D/E + rule-based tips
03 AI assistant	Income, expenses, savings, borrowed, owned; question	Savings ratio & leverage insight; keyword glossary answers
04 Loan insights	Table of applicants	Per-row credit insight and recommendation
05 Credit & risk	Score 300–900, income, EMI	Category (Poor...Excellent); risk band raised one level if EMI/income > 40%
06 Investment call	Revenue growth, margin, ROA, ROE	1 point each for ≥ 15/15/10/15 → BUY (≥ 3), HOLD (2), AVOID
07 Company perf.	Revenue growth, CSAT, retention	34/33/33 points for > 15%, > 80%, > 85% → Excellent/Good/Needs improvement
08 Portfolio	Ratios or raw financials per company	Weighted score (ROE 30, ROA 25, growth 25, margin 20), normalised to available data
09 Goals	Target, years, return, inflation, savings, income	FV, monthly SIP, year-wise corpus, affordability vs income
12 Planner	Amount, prices, days; entry, target %, stop %	Value, gain, CAGR; target/stop prices and risk–reward ratio
13 Tax tools	Prices, dates, quantity; monthly prices	STCG/LTCG with cess; SIP vs lump sum; 52-week position

4.2.1 Credit Score and Customer Risk

Scores below 550 are “Very High Risk”, 550–649 “High”, 650–749 “Medium”, 750–799 “Low” and 800+ “Very Low”. When both income and EMI are supplied and EMI exceeds 40% of income, the band moves one step riskier and the explanation states this explicitly (Figure 4.1).

Field	Value
Credit Score	780
Result	Very Good - Low Risk
Insight	Good credit score. Eligible for loan approval.

Field	Value
Name	R. Verma
Income (₹/mo)	50000
EMI (₹/mo)	22000
Credit Score	700
Result	R. Verma: High Risk
Insight	Credit score 700.0 places this in the base risk band shown in the table. EMI is 44% of income. That repayment burden pushed the risk level up by one step.

Figure 4.1: Credit score lookup and customer risk assessment

4.2.2 Portfolio Screener

The screener accepts either ratios or raw financial statement values for each company; missing ratios are derived (e.g. ROE from net income and equity). Only criteria with data contribute to the denominator, so a company is not penalised for a missing field. Scores ≥ 75 map to BUY/low risk, ≥ 50 to HOLD/medium risk and below that to SELL/high risk (Figure 4.2).

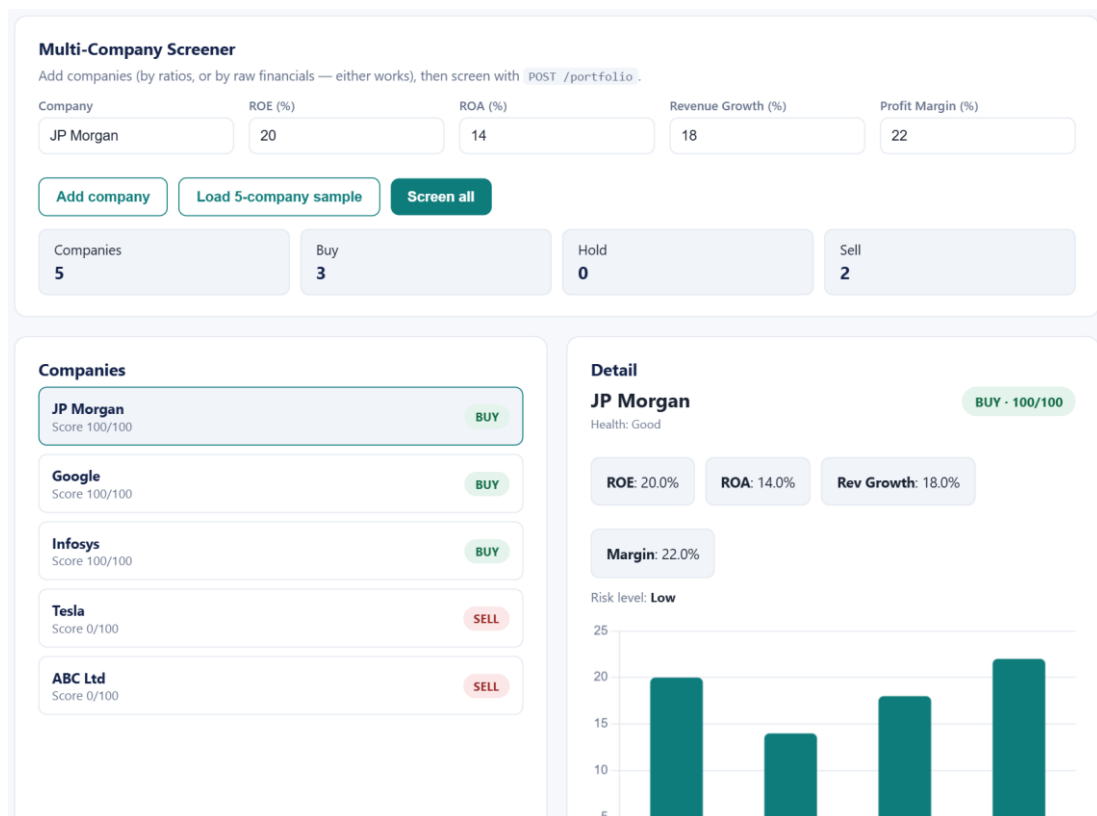


Figure 4.2: Multi-company portfolio screener with per-company detail

4.2.3 Goal Planner

The planner applies the formulas of Section 2.3 and produces a year-by-year projection of the corpus against the inflation-adjusted target. For a ₹50 lakh goal in 15 years with ₹2 lakh saved, 12% expected return and 6% inflation, the target grows to about ₹1.20 crore and the required SIP is about ₹21,579 per month — 27% of an ₹80,000 monthly income, which the insight flags as manageable (Figure 4.3).

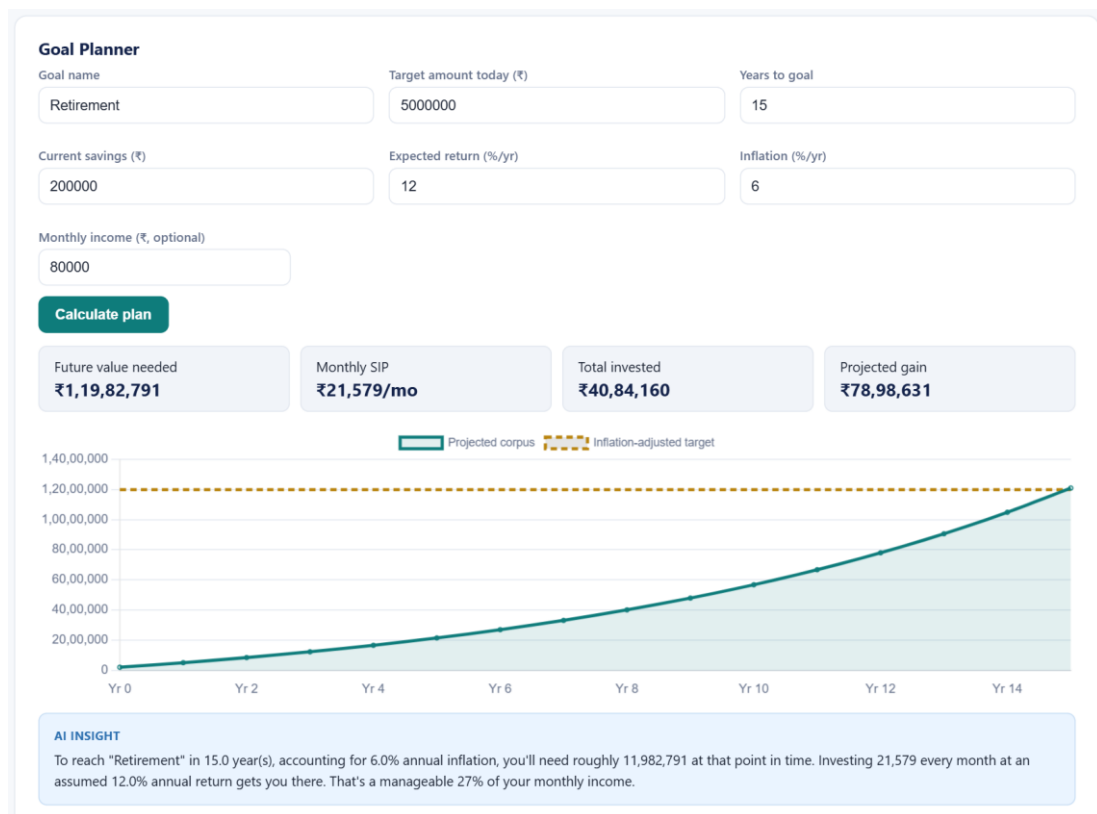


Figure 4.3: Goal planner: required SIP and projected corpus versus inflation-adjusted target

4.2.4 Capital Gains Tax and Related Tools

The tax estimator computes the holding period, classifies the gain as STCG (≤ 365 days, 20%) or LTCG (12.5% above the remaining ₹1.25 lakh exemption, after accounting for other LTCG in the year), adds 4% cess and reports net proceeds. The same tab offers a SIP-versus-lump-sum comparison and a 52-week range check.

SIP vs Lump Sum

Total to invest (₹)
Months
Start price (₹)
Last price (₹)

60000
12
2500
2800

Monthly prices, comma-separated (one per month, oldest first)

2500,2520,2480,2600,2550,2610,2650,2700,2680,2720,2750,2800

Compare

Lump sum value: ₹67200.00
SIP value: ₹63969.57
Winner: **Lump Sum** by ₹3230.43

Capital Gains Tax (India, simplified)

Buy price (₹)
Sell price (₹)
Quantity

2500
2800
10

Buy date
Sell date
Other LTCG this year (₹)

01 - 01 - 2024
01 - 06 - 2025
0

Estimate

Holding period: **517 days** → **LTCG (12.5%)**
Gross gain: ₹3000.00
Estimated tax (incl. cess): ₹0.00
Net proceeds: ₹28000.00

52-Week Range Check

Current price (₹)
52-week high (₹)
52-week low (₹)

2650
3100
2200

Check

Figure 4.4: Portfolio and tax tools

4.3 Live Market-Data Layer

The market layer (`app/market.py`) is the most engineering-intensive part of the project. Its design goals were: support every listed instrument, accept whatever the user types, never require a key, and stay fast and reliable despite depending on free upstream services.

4.3.1 Company Search

`GET /market/search` queries Yahoo Finance’s search through `yfinance`, keeps equities, ETFs, indices and mutual funds, and re-ranks results so NSE listings appear first, then BSE, then other exchanges, with block-deal series last — while otherwise preserving Yahoo’s relevance order. Searching “tata motors”, “samsung” or “nifty” therefore returns `TMCV.NS`, `005930.KS` and `^NSEI` respectively as top results.

4.3.2 Symbol Resolution

Users may type a company name, a bare ticker or a full symbol. The resolver yields candidates in order: input that looks like a name (contains spaces or lower-case letters) is searched first; ticker-like input is tried directly, then with `.NS` and `.BO` suffixes, with search as the last resort. Legacy symbols such as `RELIANCE.BSE` from the earlier Alpha Vantage version are mapped to their NSE equivalents. A wrong guess fails after a single daily-history request, keeping invalid look-ups fast.

4.3.3 Quotes and History

A quote combines one month of daily bars (which establish the previous close and confirm the symbol exists) with the last five days of one-minute bars, restricted to the latest session. This gives the last traded price, day open/high/low, volume and the exact time of the last trade, and works correctly on weekends and exchange holidays — during development, 2 October 2026 was an NSE holiday and the quote correctly showed the 1 October session. The currency is read from the response metadata so that prices are labelled ₹, \$, KRW, etc.

4.3.4 Caching and Fault Tolerance

Table 4.2: In-memory TTL cache configuration

Data	Cache lifetime	Rationale
Quote	30 seconds	Matches the auto-refresh interval; absorbs watchlist bursts
Daily history	10 minutes	Changes at most once per session
News	5 minutes	Headlines change slowly; avoids hammering the RSS feed
Search results	1 hour	Listings rarely change
Company names	24 hours	Static reference data

If a live call fails, the last good value is served instead of an error. Google News occasionally returns a transient 404 for valid queries, so requests are retried up to three times with back-off.

4.3.5 Relevant News

A naive full-text search for a company sorted by date surfaced pages that merely mention the company — index pages, rich lists, other companies' quote pages. The final algorithm (i) converts a ticker into keywords: the company's legal name without suffixes plus, where it differs, the ticker as a brand alias (Avenue Supermarts + DMart, Tata Consultancy Services + TCS); (ii) searches headlines with `intitle:` operators over the last seven days;

(iii) keeps only headlines that contain a keyword; (iv) tops up from a looser search if fewer than ten remain; and (v) removes syndicated duplicates. After this change, all ten DMart headlines in testing named the company, compared with only about half before.

4.4 Frontend

The interface is a single page with sixteen tabs. A small `api()` helper wraps `fetch` and surfaces server error messages. Notable components are described below.

4.4.1 Company Picker

The company selector behaves like a dropdown with an embedded search box (Figure 4.5). Closed, it shows the selected company’s name and ticker. Opened, it lists 77 curated instruments grouped as Indices, India (NSE) and US & Global; typing filters these instantly and appends live “All markets” results from `/market/search` after a 250 ms debounce. Keyboard navigation (`↑ ↓ Enter Esc`) and click-outside are supported, and the same component is reused in the watchlist.

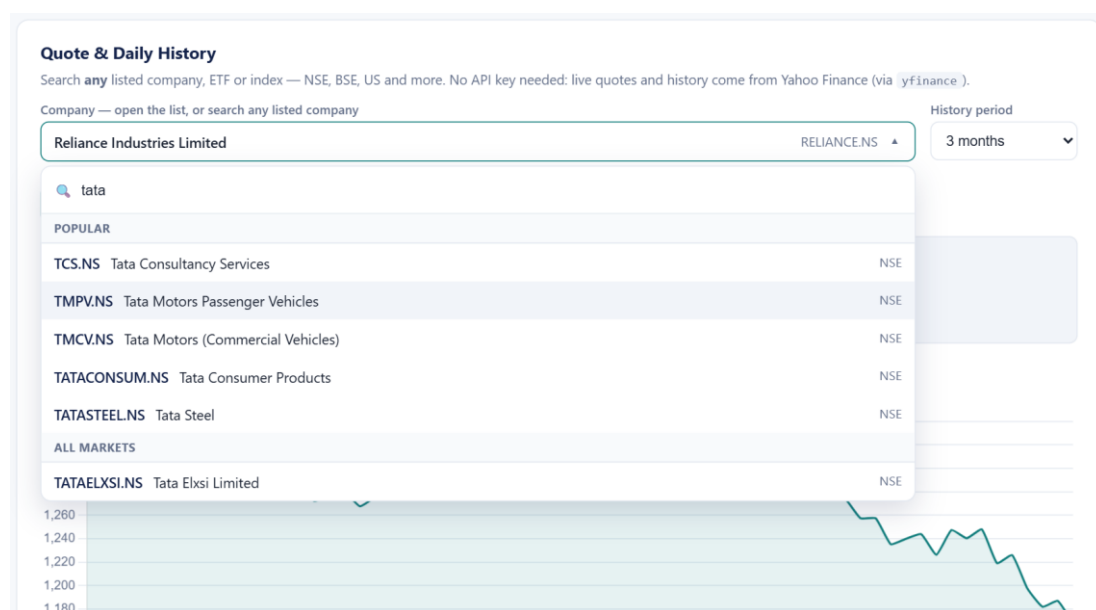


Figure 4.5: Company picker: dropdown with built-in search across all markets

4.4.2 Quote, Chart and OHLC Table

Selecting a company loads the quote, a Chart.js price chart for the chosen period (1 month to maximum) and an OHLC table (Figure 4.6). The table shows formatted dates, prices in the instrument’s currency, a day-on-day change column coloured green/red, Indian digit

grouping for volume, tooltips on each column heading and a legend explaining Open, High, Low, Close and Volume for non-expert users.

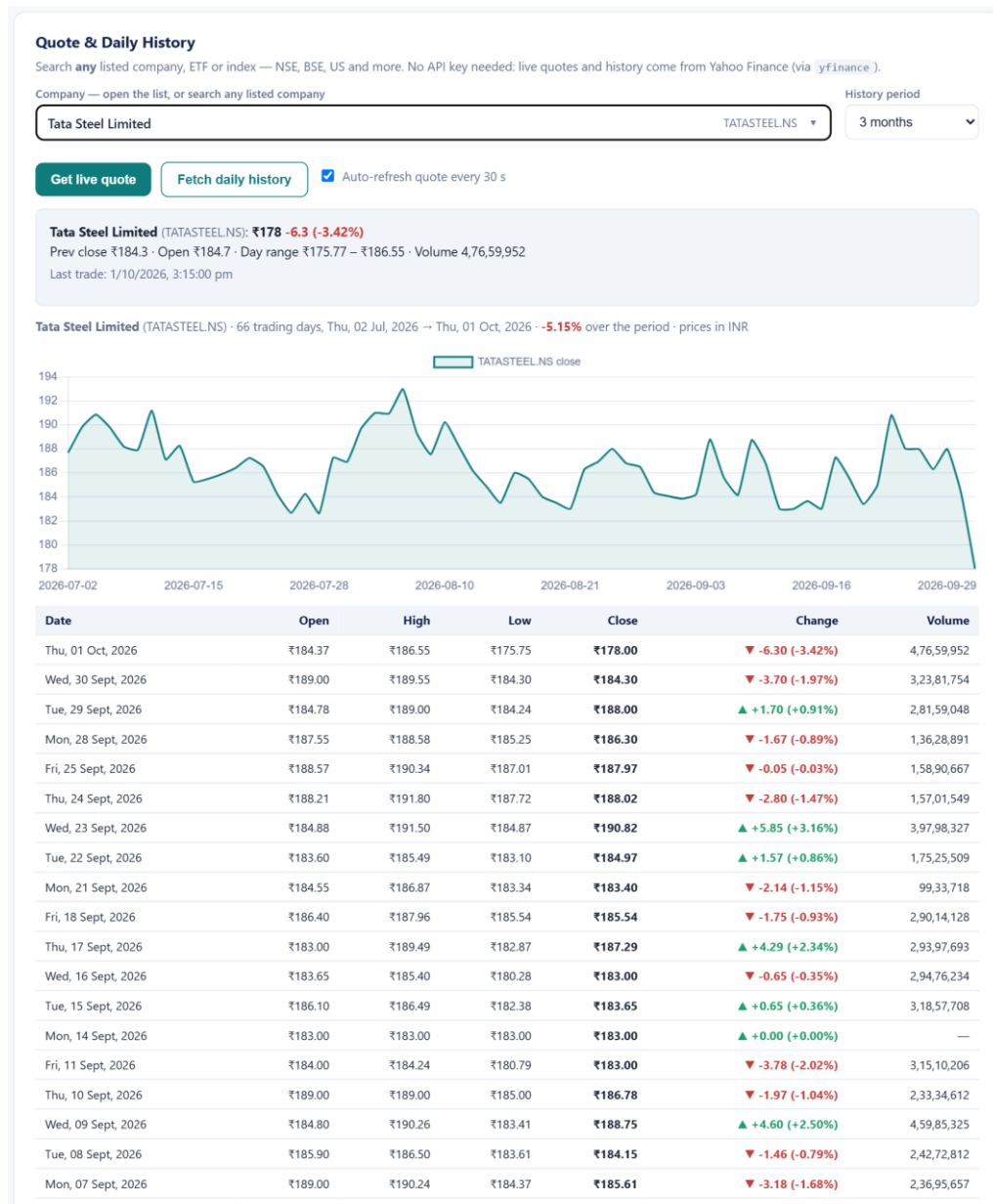


Figure 4.6: Stock market workspace: live quote, price chart and OHLC history for Tata Steel

4.4.3 News, Auto-Refresh and Consistency

News for the selected company loads automatically (Figure 4.7). A scheduler refreshes the quote every 30 seconds, history every 10 minutes and news every 5 minutes — but only while the relevant tab is open, the browser tab is visible and the user has not switched the feature off, so no requests are wasted. Each request carries a sequence number and any reply belonging to an older request is discarded, so quickly switching companies can never leave the previous company's data on screen.

News

Latest headlines from the last 7 days, newest first, via Google News' public RSS feed (no API key). Picking a company above loads its news automatically; leave the box empty for overall market news.

Tickers or topic (comma-separated)

Fetch news

Market news

☒ Auto-refresh news every 5 min

[Tata Steel receives tribunal approval for subsidiary merger](#)
Investing.com India · 9 h ago

[Tata Steel receives tribunal approval for subsidiary merger By Investing.com](#)
Investing.com Nigeria · 10 h ago

[North Eastern Carrying Corporation Ltd receives order from Tata Steel Ltd](#)
EquityBulls · 17 h ago

[NCLT approves Tata Steel merger with wholly owned subsidiary](#)
CNBC TV18 · 17 h ago

[NCLT clears Tata Steel's amalgamation with Rujuvalika Investments](#)
TipRanks · 17 h ago

Figure 4.7: Company-specific news feed

4.4.4 Watchlist and Price Alerts

Users can add any company through the same picker; prices refresh automatically and every refresh evaluates the user's "rises above / falls below" alerts (Figure 4.8).

Watchlist

No dedicated backend for this section — it reuses `GET /market/quote` from section 11 (no API key needed). Prices refresh automatically every 30 s while this tab is open, and price alerts are checked on every refresh.

AAPL

Add to watchlist

Refresh all live prices

☒ Auto-refresh every 30 s

Symbol	Price	Change	%	As of	
RELIANCE.NS Reliance Industries Limited	1167.7	-19.3	-1.63	1/10/2026, 3:15:00 pm	Remove
HDFCBANK.NS HDFC Bank Limited	721.2	12.5	1.76	1/10/2026, 3:15:00 pm	Remove
AAPL Apple Inc.	330.48	-2.54	-0.76	2/10/2026, 1:29:00 am	Remove

Price Alerts

Symbol

Alert type

Target price (₹)

Add alert

Symbol	Condition	Status
--------	-----------	--------

Figure 4.8: Live watchlist with price alerts

Chapter 5: Machine-Learning Experiment: Next-Day Price Prediction

5.1 Pipeline

The ML pipeline in `backend/services` follows four stages. **Cleaning** removes rows with a missing close and duplicate dates and reports what was removed (a synthetic data generator deliberately injects such errors for practice; the real Yahoo data used here contained none). **Feature engineering** adds daily return, MA7, MA30, price change, 21-day volatility and RSI(14) (Figure 5.1). **Dataset preparation** uses the features Close, MA7, MA30, DailyReturn and Volatility to predict the next day's close, drops rows without full history, and performs a **chronological** 80/20 split — no shuffling, so the model is always tested on data that comes after its training data. **Modelling** trains a Linear Regression on standardised features and a Random Forest with 200 trees of maximum depth 8.

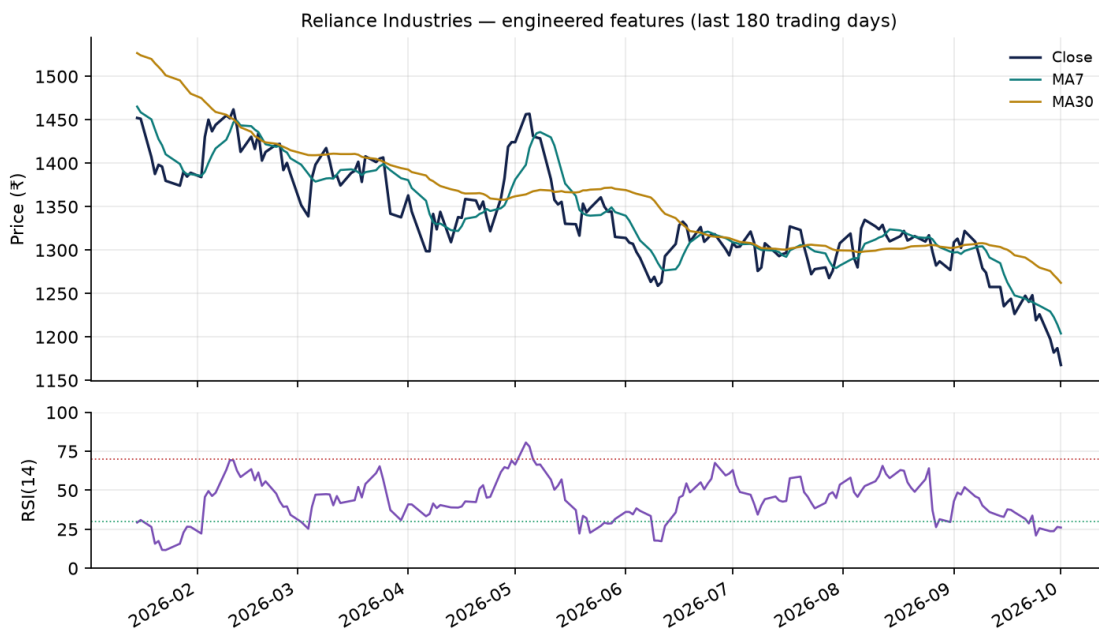


Figure 5.1: Engineered features for Reliance Industries: moving averages and RSI(14)

5.2 Experimental Setup

Two years of daily data were downloaded on 2 October 2026 for Reliance Industries, Tata Consultancy Services and Infosys. After the 30-day warm-up for MA30 and removal of the last row (no next-day target), each stock had 470 usable rows from 2024-11-13 to 2026-09-30: 376 for training and 94 for testing (test period 2026-05-22 to 2026-09-30). Models

were evaluated with MAE, RMSE and directional accuracy (whether the predicted move from today's close had the correct sign), and compared with a naive persistence forecast that predicts tomorrow's close equals today's.

5.3 Results

Table 5.1: Test-set performance (94 trading days, 2026-05-22 to 2026-09-30)

Stock	Model	MAE	RMSE	Direction correct
Reliance	Naive (tomorrow = today)	₹11.89	₹14.97	—
Reliance	Linear Regression	₹12.03	₹15.30	44.7%
Reliance	Random Forest	₹15.19	₹19.03	45.7%
TCS	Naive (tomorrow = today)	₹34.35	₹48.46	—
TCS	Linear Regression	₹34.27	₹48.33	56.4%
TCS	Random Forest	₹88.64	₹115.46	51.1%
Infosys	Naive (tomorrow = today)	₹17.07	₹23.23	—
Infosys	Linear Regression	₹17.58	₹23.43	44.7%
Infosys	Random Forest	₹39.46	₹49.94	53.2%

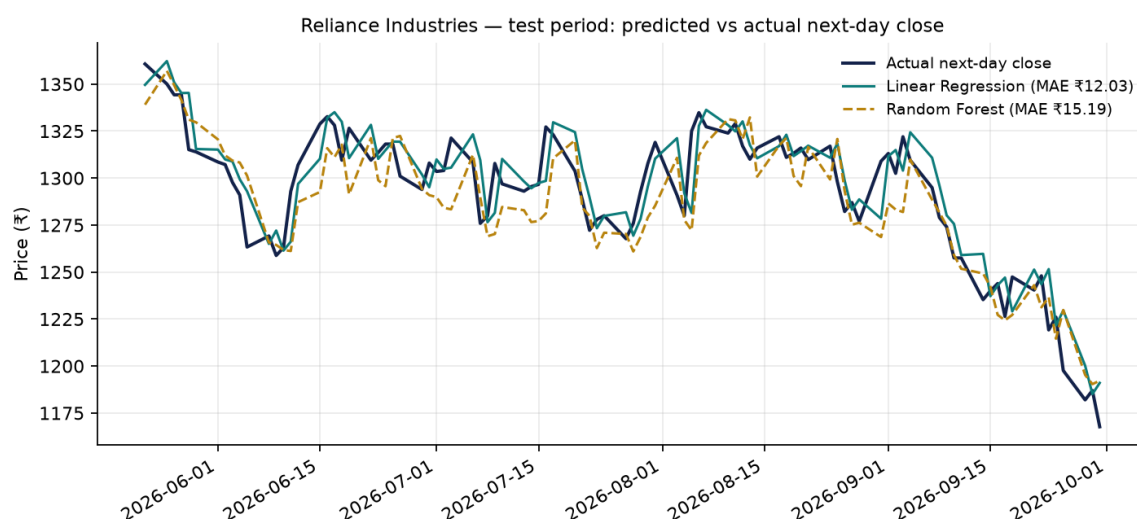


Figure 5.2: Reliance Industries: predicted versus actual next-day close on the test period

5.4 Discussion

Linear Regression matches, but does not beat, the naive baseline. For Reliance its MAE of ₹12.03 is about 0.9% of the average price — which looks impressive in isolation, yet the naive forecast achieves ₹11.89. The learned coefficients explain why: the coefficient on the standardised Close (102.2) dwarfs all others (at most 3.3), i.e. the model essentially learned to repeat today's price. Figure 5.2 shows the prediction tracking the actual series with a one-day lag.

Random Forest performs worse, especially when prices trend. Its MAE was ₹88.64 for TCS versus ₹34.35 for the baseline. Tree ensembles cannot extrapolate beyond the range of target values seen in training; when a stock moves to price levels outside that range in the test period, the forest’s predictions are capped. Its feature importances (Close 0.94 for Reliance) again show reliance on the current price.

Neither model predicts direction better than chance. Directional accuracy ranged from about 45% to 56%, close to a coin toss and not consistent across stocks. This is what the weak-form efficient market hypothesis predicts for models that use only past prices.

5.5 Implications

- Low error figures alone are misleading for price-level prediction; a persistence baseline must always be reported.
- Predicting **returns** (or direction) rather than price levels, and using models that can extrapolate, gives a fairer test.
- Additional information — fundamentals, news sentiment from the news module, market-wide factors — is required for any genuine edge; price-only technical features are insufficient.
- For these reasons the ML pipeline is presented as an educational, transparent experiment and is not exposed in the user interface as a “prediction” feature, avoiding the risk of users trading on it.

Chapter 6: Testing and Validation

6.1 Approach

Testing combined three levels: (1) direct function tests of the calculators and the market layer from Python; (2) API tests of each endpoint over HTTP, including error cases; and (3) end-to-end tests that drove the real user interface in Microsoft Edge and Google Chrome using the Playwright browser-automation library, checking on-screen values, network requests and JavaScript errors. Tests were run against live data on 1–2 October 2026.

6.2 Representative Test Cases

Table 6.1: Representative end-to-end test cases

#	Scenario	Expected	Result
T1	Quote for RELIANCE.BSE (legacy symbol)	Resolved to RELIANCE.NS with name, ₹ price, last-trade time	Pass (₹1,167.70, 1 Oct 15:15)
T2	Quote on an exchange holiday (2 Oct)	Latest session shown with day range	Pass
T3	Type “samsung” and press Get live quote	Resolved to 005930.KS, price in KRW	Pass (KRW 275,250)
T4	Bare ticker “TATASTEEL”, “WIPRO”	Resolved to .NS listing	Pass
T5	Unknown symbol “NOSUCHCO123”	HTTP 404 with guidance message	Pass
T6	Search “tata”, “hdfc”, “nifty”	Grouped suggestions incl. all-market results	Pass
T7	Daily history, period 1 year	≈ 250 rows, chart and OHLC table	Pass (251 rows for WIPRO)
T8	News for DMART.BO	Headlines naming DMart/Avenue Supermarts	Pass (10 of 10)
T9	Switch vivo → Tata Steel quickly	No stale VivoPower news shown	Pass (0 of 10 stale)
T10	Auto-refresh on market tab, then switch tab	Refresh at 30 s; none on other tabs	Pass (server log)
T11	Watchlist add via picker (Eternal, Apple, Samsung)	Live price, name, currency	Pass
T12	Empty news box → Market news	Sensex/Nifty headlines	Pass
T13	Goal plan ₹50 L, 15 y, 12% / 6%	SIP ≈ ₹21,579/month	Pass
T14	Run <code>python app/main.py</code> from another folder	Server starts on port 8000	Pass

6.3 Defects Found and Resolved

Testing on live data surfaced several real-world issues, summarised in Table 6.2. Most stem from depending on free external services and from browser caching — problems that do not appear with static demo data.

Table 6.2: Defects found during testing and their fixes

Defect	Root cause	Resolution
All market tabs returned errors	Market endpoints missing from main.py (file truncated)	Restored /market/quote, /daily, /news and /planner/what-if
Only 3 companies selectable	Hard-coded dropdown	Search endpoint + resolver supporting any listed instrument
Typed company names failed	Input treated as ticker	Name-first resolution through search
Missing day range on holidays	1-day intraday window empty	5-day window, latest session
Old UI after updates	Browser served cached JS/CSS	no-cache headers and versioned asset URLs
Irrelevant news headlines	Full-text match sorted by date	Headline keyword filter with brand aliases, de-duplication
Previous company's news shown	Out-of-order HTTP responses	Per-request sequence numbers
Intermittent news failures	Transient Google 404s	Retry with back-off; serve last good value
ModuleNotFoundError: app	Running main.py as a script	Path setup and built-in server start in main.py
₹ shown for US stocks	Hard-coded currency	Currency from Yahoo metadata

Chapter 7: Limitations and Future Work

7.1 Limitations

- **Data source reliability:** yfinance uses unofficial Yahoo Finance endpoints that may change or throttle without notice; prices for NSE/BSE may be delayed by a few minutes and are not suitable for trading.
- **Unlisted companies:** private companies (e.g. the smartphone maker Vivo) have no market data; search then returns the closest listed name.
- **Rule-based “AI” assistant:** the assistant uses keyword matching over a curated glossary, not a language model.
- **Simplified rules:** scoring thresholds are generic heuristics; the tax module ignores surcharge, grandfathering and loss set-off.
- **State:** watchlists and alerts live in the browser session and are lost on reload; there are no user accounts.
- **ML scope:** the prediction pipeline is offline and, as shown in Chapter 5, has no demonstrated predictive edge.

7.2 Future Work

- Persist watchlists and alerts (local storage or a small database) and add notifications.
- Expose the ML pipeline in the UI as an educational “model lab” that always displays the naive baseline beside model results.
- Predict returns/direction with walk-forward validation and add fundamental and news-sentiment features.
- Integrate a large language model for the assistant, grounded in the platform’s own calculations and glossary.
- Add an automated test suite (pytest for calculators, mocked market data, Playwright UI tests in CI).
- Package for deployment (Docker, HTTPS hosting) and add an official exchange data feed for production use.

Chapter 8: Conclusion

This project delivered FinSuite, an integrated finance platform that places sixteen analytical tools — from ratio analysis and credit-risk screening to SIP planning, capital-gains tax estimation and live market tracking — behind a single, transparent interface backed by a documented REST API. Each calculation explains its formula and reasoning, aiming to improve users' understanding rather than simply produce a number.

The live market-data layer meets the goal of covering any listed company worldwide without paid keys. Achieving this reliably required more than calling a library: symbol resolution for names and bare tickers, ranking for the Indian market, holiday-aware quotes, caching, retries, relevance-filtered news and protection against stale browser data were all necessary, and each was driven by a defect observed while testing with real data.

The machine-learning experiment produced an equally important, if less glamorous, result: price-only models with low headline error did not beat the naive forecast, and could not predict direction. Reporting this honestly, with a proper baseline, is itself a contribution — it demonstrates how AI in finance should be evaluated, and why responsible tools should not present such models to users as reliable predictions.

Overall, the project shows that a carefully engineered, free and open platform can bring professional-style financial analysis within reach of students, first-time investors and small savers, and provides a solid foundation for the extensions outlined in Chapter 7.

References

- [1] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
- [2] Brealey, R. A., Myers, S. C., & Allen, F. (2020). *Principles of Corporate Finance* (13th ed.). McGraw-Hill Education.
- [3] Fama, E. F. (1970). Efficient Capital Markets: A Review of Theory and Empirical Work. *The Journal of Finance*, 25(2), 383–417.
- [4] Wilder, J. W. (1978). *New Concepts in Technical Trading Systems*. Trend Research.
- [5] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [6] McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61.
- [7] Government of India. Income-tax Act, 1961 — Sections 111A and 112A, as amended by the Finance (No. 2) Act, 2024.
- [8] TransUnion CIBIL. CIBIL Score and Report — consumer information.
<https://www.cibil.com>
- [9] FastAPI documentation. <https://fastapi.tiangolo.com>
- [10] yfinance — Download market data from Yahoo! Finance’s API.
<https://github.com/ranaroussi/yfinance>
- [11] Chart.js documentation. <https://www.chartjs.org/docs/>
- [12] Playwright for Python documentation. <https://playwright.dev/python/>

Appendix A: API Reference

Table A.1: REST endpoints

Method	Endpoint(s)	Purpose
POST	/ratios/roe, /ratios/roa, /ratios/debt-ratio, /ratios/health	01 Ratio calculators
POST	/report	02 Personal finance report
POST	/assistant/snapshot, /assistant/chat	03 AI finance assistant
POST	/loans/screen	04 Loan insight generator
POST	/credit-score, /risk	05 Credit score & risk
POST	/invest	06 Investment recommendation
POST	/company-performance	07 Company performance
POST	/portfolio	08 Portfolio dashboard
POST	/goals/plan, /goals/emergency-fund	09 Goal & retirement planner
GET	/knowledge/glossary	10 Knowledge centre
GET	/market/search?q=	11 Find any company/ETF/index
GET	/market/quote?symbol=	11 Live quote (name, price, change, range, volume, currency, last trade)
GET	/market/daily?symbol=&period=	11 Daily OHLCV (1mo–max) with resolved ticker, name, currency
GET	/market/news?symbols=&limit=	11 Company or market news (last 7 days)
POST	/planner/what-if, /planner/target-plan	12 Growth & target planner
POST	/tools/sip-vs-lumpsum, /tools/capital-gains-tax, /tools/52-week-range	13 Portfolio & tax tools
GET	/api-keys	15 API key directory (none required)
GET	/about, /api/status, /docs, /	16 About, health check, Swagger UI, web app

Appendix B: Installation and Running

1. Install Python 3.11 or later.
2. In the project folder create and activate a virtual environment: `python -m venv agentic_env` then `agentic_env\Scripts\activate` (Windows).
3. Install dependencies: `pip install -r requirements.txt`.
4. Start the server: `python app/main.py` (or `uvicorn app.main:app --reload`).
5. Open <http://127.0.0.1:8000> for the application and <http://127.0.0.1:8000/docs> for the interactive API documentation.
6. Optional: run `python -m app.market TCS` to test the market-data layer from the command line.

Appendix C: Application Home Screen

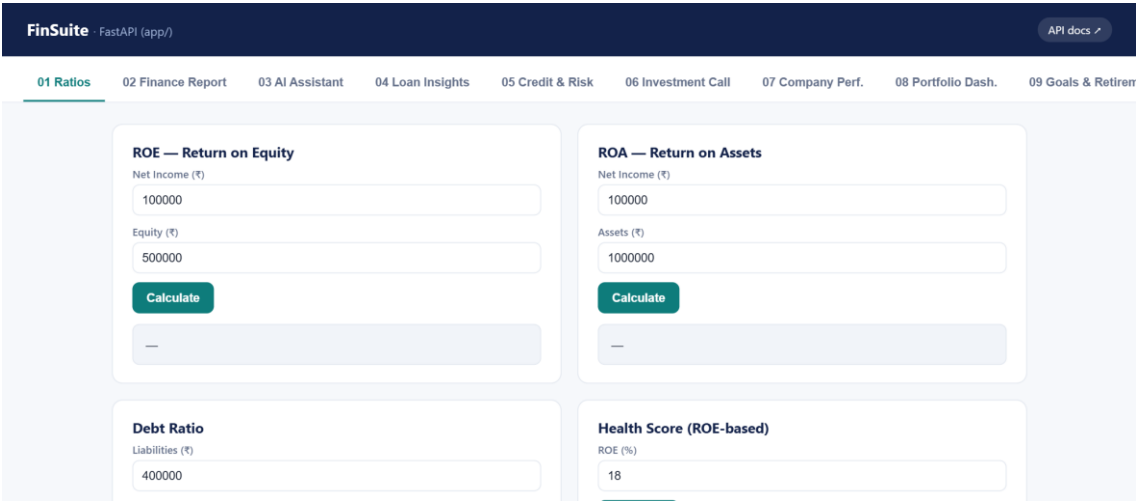


Figure A.1: FinSuite home screen and module navigation

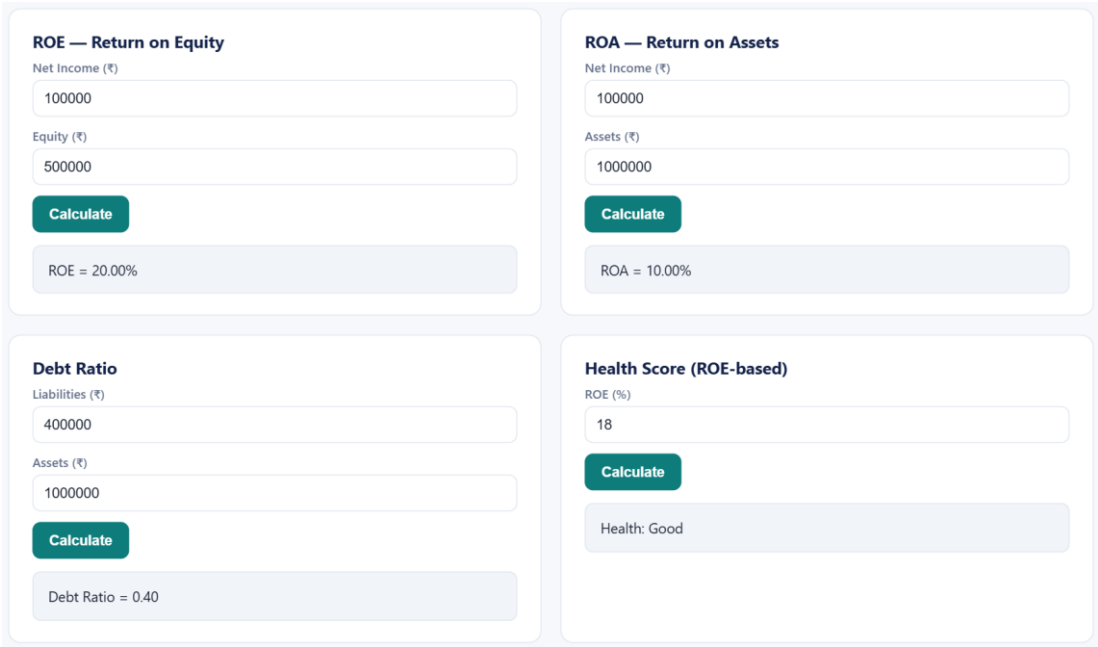


Figure A.2: Module 01 — financial ratio calculators